Pairion Scene Architecture

Version: 1.0 **Date:** 2026-04-21 **Status:** Foundational **Purpose:** Defines the plugin-based contextual scene system using MVC separation.

---

1. Overview

Pairion's screen is a contextual computing surface. The entire background morphs based on what the user is discussing with Jarvis. Ask about weather → globe zooms in. Ask about flights → ATC radar appears. Ask about stocks → trading terminal loads. Ask about directions → street map with route.

This is implemented as a plugin architecture where anyone can build a scene without touching Pairion's core codebase. A scene is a pure QML View that receives structured data from the server and renders it. Plugin authors write only the View. Pairion handles everything else.

---

2. MVC Separation

Layer	Responsibility	Who writes it	Where it lives
Model	Fetch external data, transform it, push to client	Server contributors	`Pairion-Server/pairion-adapters/` data model packages
View	Render data visually. Pure QML. No HTTP, no business logic	Plugin authors (anyone)	`~/.pairion/scenes/` or `pairion-scenes` community repo
Controller	Scene lifecycle, data routing, LLM tool dispatch, transitions	Core Pairion team	`Pairion-Client` SceneManager + `Pairion-Server` ToolDispatcher

---

Strict boundaries:

- A **View** (plugin) NEVER makes HTTP calls, opens sockets, or accesses the filesystem.
- A **View** receives data as QML properties and renders it. That is its only job.
- A **Model** (data adapter) NEVER knows which View will consume its data.
- A **Model** produces a documented data schema. Any View can bind to it.

- The **Controller** connects Models to Views and manages scene transitions. Plugin authors never touch it.

---

3. Plugin Structure

A scene plugin is a self-contained directory with a manifest and a QML entry point.

3.1 Directory Layout

```
~/.pairion/scenes/
├── adsb-radar/
│   ├── scene.yaml              ← manifest (required)
│   ├── AdsbRadarScene.qml      ← entry point (required, named in manifest)
│   ├── RadarSweep.qml          ← internal QML components (optional)
│   ├── AircraftBlip.qml
│   └── assets/                 ← images, fonts, shaders (optional)
│       ├── sweep.png
│       └── blip.png
├── stock-ticker/
│   ├── scene.yaml
│   ├── StockTickerScene.qml
│   ├── CandlestickChart.qml
│   └── TickerBar.qml
└── recipe-browser/
    ├── scene.yaml
    ├── RecipeBrowserScene.qml
    └── assets/
        └── icons/
```

Built-in scenes (shipped with Pairion) live in the application bundle under `qml/Scenes/` and follow the same structure and manifest format. They are not special — they just ship pre-installed.

3.2 Manifest (`scene.yaml`)

```yaml
```

```yaml
# === Required fields ===

# Unique identifier. Lowercase, hyphenated. Must match directory name.
id: adsb-radar

# Human-readable display name.
name: ADS-B Radar

# SemVer version string.
version: 1.0.0

# QML file name of the root scene component (relative to this directory).
entry: AdsbRadarScene.qml

# === Optional fields ===

# Author information.
author: Adam Allard
license: Apache-2.0
description: Real-time aircraft tracking radar using ADS-B data
url: https://github.com/aallard/pairion-scene-adsb

# Category for organization and discovery.
# Suggested values: aviation, finance, communication, navigation,
#                   home, food, weather, media, productivity, utility
category: aviation

# Minimum Pairion version required.
min_pairion_version: 0.4.0

# Data models this scene consumes.
# The server must have a matching data model adapter registered.
# Each entry specifies a data model ID and optional configuration.
data_models:
  - id: adsb
    config:
      radius_nm: 15
      center: home    # "home" = use household coordinates

# Conversational triggers.
# When the user says something matching these phrases, the LLM considers
# activating this scene. These are hints, not exact matches — the LLM
# uses its judgment based on conversational context.
```

```yaml
triggers:
  - aircraft nearby
  - planes overhead
  - flight tracker
  - what's flying
  - aviation radar
  - ADS-B

# LLM tool definition.
# Registered automatically when the scene is loaded.
# The LLM can call this tool to activate the scene with parameters.
tool:
  name: show_adsb_radar
  description: Display real-time aircraft radar showing nearby flights with altitude
  parameters:
    radius_nm:
      type: number
      description: Radar radius in nautical miles
      default: 8
    center_lat:
      type: number
      description: Center latitude (defaults to home)
    center_lon:
      type: number
      description: Center longitude (defaults to home)

# Preview image for scene browser / marketplace (optional).
preview: assets/preview.png
```

3.3 Entry Point QML Component Contract

Every scene entry point QML file must conform to this interface:

```
// AdsbRadarScene.qml
import QtQuick

Item {
    id: root

    // === REQUIRED: Pairion injects these properties ===

    // The full width and height of the scene area (entire screen minus
    // persistent HUD overlays if any). The scene fills this space.
    // Provided by SceneManager.
    property real sceneWidth: 0
    property real sceneHeight: 0

    // Data from the declared data_models in scene.yaml.
    // Key = data model ID from the manifest.
    // Value = JS object/array conforming to the data model's schema.
    // Updated in real-time as new data arrives from the server.
    property var sceneData: ({})

    // Parameters passed when the scene was activated.
    // These come from the LLM tool call's arguments.
    property var sceneParams: ({})

    // Current HUD state (idle/listening/thinking/speaking/error).
    // Scenes can optionally react to this (e.g., dim during idle).
    property string hudState: "idle"

    // === REQUIRED: Scene must set these ===

    // Fill the available space.
    anchors.fill: parent

    // === OPTIONAL: Scene can emit these signals ===

    // Request the controller to switch to a different scene.
    // Example: a recipe scene might link to a grocery list scene.
    signal requestScene(string sceneId, var params)

    // Notify the controller that the scene wants to speak something.
    // The controller routes this to the TTS pipeline.
    signal requestSpeak(string text)
```

```
    // === Scene implementation ===

    // Access data like this:
    // sceneData.adsb → array of aircraft objects (per data model schema)
    // sceneParams.radius_nm → parameter from LLM tool call
}
```

3.4 What Plugin Authors Do NOT Need To Do

- Write any C++ code
- Understand the build system
- Handle WebSocket connections
- Make HTTP requests for data
- Register tools with the LLM
- Manage scene transitions
- Handle audio or voice pipeline
- Write tests for Pairion internals (they test their own QML)

---

4. Data Models

4.1 Concept

A data model is a server-side adapter that fetches, transforms, and pushes structured data to the client over WebSocket. Each data model has a unique ID, a documented schema, and a refresh strategy.

Data models are independent of scenes. Multiple scenes can consume the same data model. A stock chart scene and a portfolio dashboard scene both consume the `stock_quotes` data model.

4.2 Data Model Registry

The server maintains a registry of available data models. Each data model implements a common interface:

```
DataModelAdapter (SPI interface)
├── id(): String                          → "adsb"
├── schema(): DataModelSchema             → documents the output fields
├── configure(Map<String, Object>): void  → receives config from scene manifest
├── start(): void                         → begin producing data
├── stop(): void                          → stop producing data (when no scene
needs it)
├── currentData(): Map<String, Object>    → latest snapshot
```

4.3 Data Flow

```
External API  →  DataModelAdapter (server)
                        |
                        ▼
              SceneDataPush (WebSocket message)
              { type: "SceneDataPush", modelId: "adsb", data: [...] }
                        |
                        ▼
              SceneManager (client)
                        |
                        ▼
              sceneData.adsb = [...] (QML property binding)
                        |
                        ▼
              Plugin QML renders it
```

4.4 Refresh Strategies

Strategy	Use case	Example
Poll	Periodic HTTP fetch	ADS-B (every 5s), stock quotes (every 15s)
Push	Server-initiated updates	Home Assistant state changes, email notifications
On-demand	Fetched once when scene activates	Weather forecast, recipe search results
Streaming	Continuous real-time stream	Live market data, audio waveform

The refresh strategy is defined in the data model adapter, not in the scene manifest. The scene just sees updated `sceneData` properties — it doesn't know or care how often data refreshes.

4.5 Planned Data Models

ID	Source	Schema (key fields)	Refresh
`adsb`	ADS-B Exchange API or local dump1090	`[{icao, callsign, lat, lon, alt_ft, speed_kts, heading, squawk}]`	Poll 5s
`stock_quotes`	Alpha Vantage / Yahoo Finance	`[{symbol, price, change, change_pct, volume, high, low}]`	Poll 15s
`stock_history`	Alpha Vantage / Yahoo Finance	`{symbol, interval, candles: [{t, o, h, l, c, v}]}`	On-demand
`gmail_inbox`	Gmail API (OAuth)	`[{id, from, subject, snippet, date, unread}]`	Push (via poll 60s)
`gmail_message`	Gmail API (OAuth)	`{id, from, to, subject, body_html, date, attachments}`	On-demand
`weather_current`	Open-Meteo	`{temp_f, conditions, wind_mph, humidity, city}`	On-demand
`weather_forecast`	Open-Meteo	`{city, daily: [{date, high, low, conditions}]}`	On-demand
`news_headlines`	NewsAPI or RSS	`[{title, source, url, published, lat, lon}]`	Poll 300s
`ha_entities`	Home Assistant API	`[{entity_id, state, attributes, last_changed}]`	Push
`osm_route`	OSRM or GraphHopper	`{distance_mi, duration_min, steps: [{instruction, distance, geometry}]}`	On-demand
`osm_tiles`	OpenStreetMap tile server	Tile URLs for map rendering	On-demand
`recipes`	Local database	`[{id, name, time_min, ingredients, steps, image}]`	On-demand

ID	Source	Schema (key fields)	Refresh
`pantry`	Local database	`[{item, quantity, unit, expiry}]`	On-demand
`garden`	Local database	`{beds: [{name, plants: [{name, planted, status}]}]}`	On-demand
`calendar`	Google Calendar API (OAuth)	`[{title, start, end, location, description}]`	Push (via poll 120s)
`homestead`	Composite (HA + weather + local)	`{weather, animals, garden_summary}`	Push

5. Controller Layer

5.1 Server-Side: Scene Orchestration

SceneRegistry — scans `~/.pairion/scenes/` at startup, parses every `scene.yaml`, registers:

- Tools with the ToolDispatcher (from each scene's `tool` block)
- Trigger phrases with the LLM system prompt (from each scene's `triggers` block)
- Data model requirements (from each scene's `data_models` block)

SceneOrchestrator — manages active scene state per session:

- Receives `set_scene(sceneId, params)` tool calls from the LLM
- Sends `SceneChange` WebSocket message to the client
- Starts/stops data model adapters as scenes activate/deactivate
- Pushes data model updates via `SceneDataPush` WebSocket messages

LLM Integration — the system prompt includes:

- List of available scenes with their trigger descriptions
- A `set_scene` meta-tool that the LLM uses to switch contexts
- Instructions: "When the conversation shifts to a topic that matches a scene's triggers, call set_scene to activate it. When the topic shifts away, call set_scene with 'dashboard' to return to the default."

5.2 Client-Side: SceneManager

SceneManager (QML component, replaces/expands `ContextBackground.qml`):

- Receives `SceneChange` messages via ConnectionState
- Dynamically loads the target scene's entry QML file using `Qt.createComponent()`
- Injects `sceneWidth`, `sceneHeight`, `sceneData`, `sceneParams`, `hudState` properties
- Manages scene transitions (crossfade, slide, or custom per-scene)
- Routes incoming `SceneDataPush` messages to the active scene's `sceneData` property
- Maintains a scene cache (keeps recently used scenes in memory for fast switching)
- Falls back to the built-in dashboard scene if a scene fails to load

5.3 WebSocket Protocol Additions

```
SceneChange (Server → Client)
{
    "type": "SceneChange",
    "sceneId": "adsb-radar",
    "params": { "radius_nm": 8 },
    "transition": "crossfade"    // optional: crossfade, slide, instant
}

SceneDataPush (Server → Client)
{
    "type": "SceneDataPush",
    "modelId": "adsb",
    "data": [ { "icao": "A12345", "callsign": "DAL1234", ... } ]
}

SceneClear (Server → Client)
{
    "type": "SceneClear"         // return to default dashboard
}
```

---

6. Persistent HUD Overlay

These elements render ON TOP of every scene and never change when scenes switch:

Element	Position	Purpose
RingSystem	Center	Visual state indicator (idle/listening/thinking/speaking)
TranscriptStrip	Bottom	Live STT transcripts
TopBar	Top	World city clocks
FpsCounter	Top-right (dev only)	Performance monitoring

The plugin scene renders behind these elements. The scene's `sceneWidth` and `sceneHeight` represent the full screen area — the scene is responsible for not placing critical content behind the ring system or transcript strip (though slight overlap is fine for atmospheric backgrounds).

---

7. Built-In Scenes

These ship with Pairion and live in `qml/Scenes/` (same manifest format as plugins):

Scene ID	Entry QML	Description
`dashboard`	DashboardScene.qml	Default idle scene. Clocks, headlines, inbox summary, todos, homestead. The ambient home screen.
`globe`	GlobeScene.qml	3D interactive globe with news pins, weather overlay, location focus. Current implementation refactored into this structure.
`space`	SpaceScene.qml	Animated star field. Activated during astronomy/space conversations. Current implementation refactored.

All other scenes (ADS-B, stocks, email, recipes, etc.) are plugins — they can ship in a separate `pairion-scenes` repo and be installed independently.

---

8. Scene Lifecycle

```
1. DISCOVERY
   Server startup → scan ~/.pairion/scenes/ → parse scene.yaml files
   → register tools + triggers + data model requirements

2. ACTIVATION
```

```
   User speaks → LLM decides scene is relevant → calls set_scene tool
   → Server sends SceneChange to client
   → SceneManager loads QML, injects properties, starts transition

3. DATA BINDING
   Server starts required data model adapters → adapters fetch/stream data
   → Server pushes SceneDataPush messages
   → SceneManager routes data to active scene's sceneData property
   → QML property bindings update the View automatically

4. INTERACTION
   Scene renders, animates, responds to hudState changes
   Scene can emit requestScene() to chain to another scene
   Scene can emit requestSpeak() to make Jarvis say something

5. DEACTIVATION
   Conversation shifts → LLM calls set_scene("dashboard") or set_scene(different)
   → SceneManager starts exit transition on current scene
   → Server stops data model adapters that no other scene needs
   → New scene loads and activates

6. ERROR HANDLING
   If scene QML fails to load → SceneManager logs error, falls back to dashboard
   If data model adapter fails → sceneData property contains error field,
   scene can display a graceful "data unavailable" state
```

---

9. Plugin Development Experience

For a plugin author who wants to build a new scene:

1. Create a directory: `~/.pairion/scenes/my-scene/`
2. Create `scene.yaml` with the manifest fields
3. Create `MyScene.qml` implementing the entry point contract
4. Restart Pairion — the scene auto-registers
5. Say the trigger phrase — the scene loads

What they DON'T need:

- Pairion source code
- C++ compiler

- CMake / Maven / any build system
- API keys (data model adapters handle authentication)
- WebSocket knowledge
- Understanding of the audio pipeline

What they DO need:

- QML/JavaScript knowledge
- The data model schema documentation (published on pairion.dev)
- A text editor

---

10. Future: Marketplace (M14)

The plugin architecture directly supports a marketplace:

- `pairion install scene adsb-radar` → downloads from registry, drops into `~/.pairion/scenes/`
- `pairion list scenes` → shows installed + available
- `pairion update scenes` → checks for new versions
- `pairion remove scene adsb-radar` → deletes the directory

The marketplace is a curated index of scene.yaml manifests pointing to Git repos. Installation is `git clone` into the scenes directory. No package manager complexity needed.

---

11. Security Considerations

Current (self-hosted, single household):

- Plugins run in the QML context with full access to ConnectionState and other QML singletons
- Acceptable because users install their own plugins on their own hardware
- Data model adapters on the server have full network access (they need it to fetch APIs)

Future (marketplace, community plugins):

- QML sandboxing: restrict plugin access to only `sceneData`, `sceneParams`, `hudState`, `sceneWidth`, `sceneHeight`

- Data model sandboxing: restrict which URLs a data model adapter can reach
- Plugin signing: verify plugin authenticity before installation
- Review process: community review before listing in marketplace

These are M14+ concerns. Not blocking for initial implementation.

---

12. Relationship to Existing Code

Current file	Becomes	Notes
`ContextBackground.qml`	`SceneManager.qml`	Expa from earth toggl scene lifecy mana
`HemisphereMap.qml`	`qml/Scenes/globe/GlobeScene.qml`	Refac into a plugi follov entry contr
`DashboardPanels.qml` + panels	`qml/Scenes/dashboard/DashboardScene.qml`	Refac into t defau scene
`ConnectionState.backgroundContext`	`ConnectionState.activeSceneId` + `ConnectionState.sceneData`	Expa from to sce data
`MapFocusTool` (server)	Stays, but `set_scene("globe", {lat, lon})` becomes the canonical way	`focu` can c `set_` intern
`ToolDispatcher` (server)	Unchanged — scene tools register through it	New `set_`

Current file	Becomes	Notes
		tool a along existi